FramSat-1 Downlink Packet Reception & AX.25 Framing Guide

Technical Specification for Amateur Radio Operators

Orbit NTNU – SatCom Team
Trondheim, Norway
https://orbitntnu.com

September 4, 2026

Abstract

This document provides the radio frequency (RF), modulation, and data link layer specifications for the UHF telemetry downlink of the **FramSat-1** CubeSat, developed by Orbit NTNU. It is intended to assist amateur radio operators, SatNOGS ground station owners, and enthusiasts in receiving, demodulating, and extracting raw frames transmitted by the satellite.

1 Introduction

FramSat-1 is a 1U student satellite developed by Orbit NTNU in Trondheim, Norway. The satellite transmits periodic beacon and telemetry frames on the amateur satellite 70 cm band. All transmissions adhere strictly to open amateur radio protocols, utilizing standard AX.25 UI-frames modulated with G3RUH 9600 bps GFSK.

Amateur radio operators are invited to listen for downlinks and submit reception reports to the Orbit NTNU ground operations team.

2 Physical Layer (RF & Modulation)

The physical layer parameters for the UHF downlink are summarized in Table 1.

Table 1: FramSat-1 Downlink RF Specifications

Parameter	**Value / Specification**
Downlink Frequency	**435.141 MHz** (IARU Coordinated)
Frequency Band	70 cm Amateur Satellite Service (435–438 MHz)
Modulation Scheme	GFSK (2-FSK with Gaussian filter)
Bitrate / Baudrate	9600 bps
Frequency Deviation (Δf)	± 3.0 kHz
Modulation Index (h)	≈ 0.625
Pulse Shaping Filter	Gaussian ($BT = 0.5$ / $BT = 1.0$)
Scrambling Scheme	Standard G3RUH ($1 + x^{12} + x^{17}$, Mask: `0x21`)
Bit Encoding	NRZI (Non-Return-to-Zero Inverted)
Preamble Length	50 bytes, nominal (`0x7E` / alternating synchronization bytes)
Postamble Length	7 bytes

3 Data Link Layer (AX.25 Frame Structure)

FramSat-1 utilizes standard AX.25 Unnumbered Information (UI) frames encapsulated within HDLC framing flags (`0x7E`) and verified by a 16-bit CRC-CCITT Frame Check Sequence (FCS).

The standard packet header is exactly 16 bytes long. The byte breakdown is specified in Table 2.

Table 2: AX.25 16-Byte Header Breakdown

Byte Range	Length	Field	Content / Encoding
0–5	6 B	Destination Callsign	`"CQ  "` (ASCII left-shifted $\ll 1$)
6	1 B	Destination Control	`0x60` ($C = 0, RR = 11, \text{SSID} = 0, \text{Ext} = 0$)
7–12	6 B	Source Callsign	`"LA1ORB"` (ASCII left-shifted $\ll 1$)
13	1 B	Source Control	`0x61` ($C = 0, RR = 11, \text{SSID} = 0, \text{Ext} = 1$)
14	1 B	Control Byte	`0x03` (AX.25 UI-Frame, Unnumbered Information)
15	1 B	Protocol ID (PID)	`0xF0` (No Layer 3 Protocol / Custom Telemetry)
16+	N B	Information Field	Raw Telemetry / Mission Payload

3.1 Address Bit-Shifting Rule

In accordance with the AX.25 specification, standard 7-bit ASCII characters for the callsign are left-shifted by one bit:

$$b_{\text{encoded}} = b_{\text{ASCII}} \ll 1 \tag{1}$$

Callsigns with fewer than 6 characters are space-padded with ASCII `0x20` (which encodes to `0x40`). The last bit (bit 0) of byte 13 is set to `1` to signify the end of the address field.

4 GNU Radio Companion (GRC) Receiver Pipeline

A complete demodulation chain can be built in GNU Radio Companion using standard out-of-the-box blocks. Figure 1 illustrates the recommended flowgraph utilizing a standard RTL-SDR receiver.

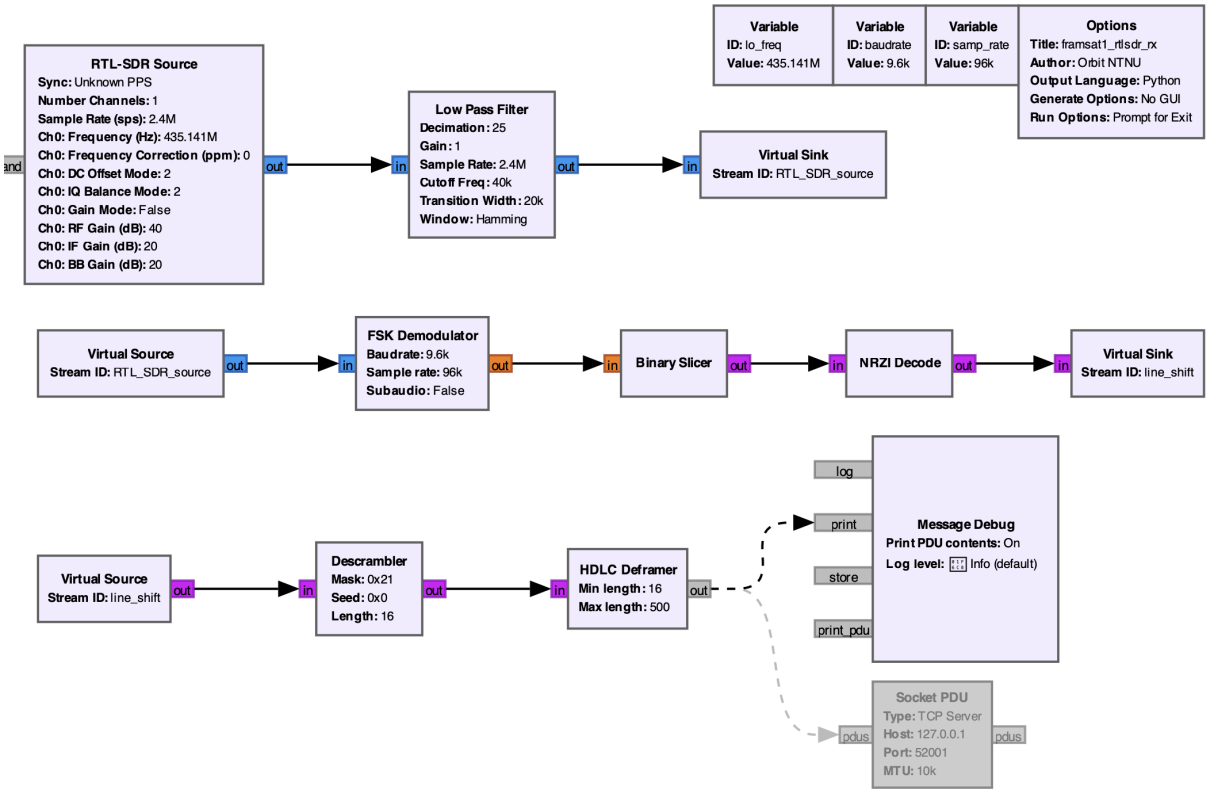

Figure 1: Recommended GNU Radio receiver pipeline for FramSat-1 using an RTL-SDR dongle (sampling at 2.4 MSps and decimated to 96 kSps).

The recommended demodulator architecture consists of:

1. **RTL-SDR Source:** Set center frequency to 435.141 MHz and sample rate to 2.4 MSps (standard stable rate for RTL2832U devices).
2. **Low Pass Filter (Decimator):** Decimates the stream by a factor of 25 (reducing the rate from 2.4 MSps to exactly 96 kSps), with a cutoff frequency of 40 kHz and transition width of 20 kHz to suppress out-of-band noise.
3. **FSK Demodulator:** Operates at the decimated sample rate of 96 kSps with baud rate set to 9600 bps.
4. **Binary Slicer:** Converts demodulated audio-frequency excursions into binary symbols (0 and 1).
5. **NRZI Decode:** Decodes bit transitions according to standard AX.25 NRZI conventions.
6. **Descrambler:** Mask: `0x21`, Seed: `0x0`, Length: `16` (Standard G3RUH polynomial).
7. **HDLC Deframer:** Minimum frame length: 16 bytes, Maximum: 500 bytes. Validates the 16-bit CRC-CCITT (FCS) and discards corrupt frames.
8. **Message Debug / Socket PDU:** Outputs valid received frames to the console via `Message Debug`, or optionally streams them to external software via `Socket PDU`.

5 Reference Python Frame Decoder

For ease of deployment, the complete, pre-configured GNU Radio Companion flowgraph (`.grc`) and the Python decoding script are maintained in an open-source repository on the Orbit NTNU GitHub organization:

https://github.com/orbitntnu/framsat1-frame-decoder

Amateur radio operators can clone or download the files directly, avoiding manual copy-pasting.

5.1 Standalone & Live Operation

Operators can view received packets directly inside GNU Radio Companion using the `Message Debug` block. Alternatively, for automated real-time decoding, the output of the `HDLC Deframer` can be connected to a `Socket PDU` block (TCP Server on port 52001).

The script in Listing 1 supports two operating modes:

- **Live TCP Mode:** Run with `--listen` to connect to a running GNU Radio flowgraph and decode frames in real time as the satellite passes overhead.
- **Offline / Inspection Mode:** Pass a hexadecimal string as a command-line argument to parse historical frames recorded by other ground stations.

```python
import sys
import socket

def parse_framsat_frame(frame_bytes: bytes):
    """Parses a raw AX.25 UI frame received from FramSat-1."""
    if len(frame_bytes) < 16:
        print("[-] Frame rejected: Length less than 16 bytes.")
        return

    # Extract callsigns (reverse the 1-bit left-shift)
    dest_call = "".join(chr(b >> 1) for b in frame_bytes[0:6]).strip()
    src_call  = "".join(chr(b >> 1) for b in frame_bytes[7:13]).strip()

    control_byte = frame_bytes[14]
    pid_byte     = frame_bytes[15]

    # Payload starts strictly at byte index 16
    payload_raw   = frame_bytes[16:]
    payload_ascii = payload_raw.decode('latin-1', errors='replace')

    print("\n" + "=" * 50)
    print(" Satellite Frame Decoded")
    print(f" Destination: {dest_call}")
    print(f" Source:      {src_call} (Expected: LA1ORB)")
    print(f" Control:     0x{control_byte:02X} (UI-Frame)")
    print(f" PID:         0x{pid_byte:02X}")
    print(f" Payload Len: {len(payload_raw)} bytes")
    print(f" Payload Text:'{payload_ascii}'")
    print(f" Payload Hex: {payload_raw.hex().upper()}")
    print("=" * 50)

def listen_live_gnuradio(host="127.0.0.1", port=52001):
    """Connects to GNU Radio's Socket PDU and decodes packets live."""
    print(f"[*] Connecting to GNU Radio Socket PDU at {host}:{port}...")
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((host, port))
        print("[+] Connected! Listening for live satellite passes...\n")
```

```
        while True:
            data = s.recv(1024)
            if not data:
                break
            parse_framsat_frame(data)
    except Exception as e:
        print(f"[-] Connection failed: {e}")
    finally:
        s.close()

if __name__ == "__main__":
    # 1. If an argument is provided, parse that hex string directly
    if len(sys.argv) > 1:
        if sys.argv[1] == "--listen":
            listen_live_gnuradio()
        else:
            raw_hex = sys.argv[1].replace(" ", "")
            parse_framsat_frame(bytes.fromhex(raw_hex))
    else:
        # 2. Default: Run test on verification vector
        print("[*] No arguments provided. Running test verification vector:")
        example_hex = "86A2404040406098826 29C8EA66103F0534354657374"
        parse_framsat_frame(bytes.fromhex(example_hex))
        print("\nUsage:")
        print("  python framsat1_decoder.py <HEX_DATA>     (Decode a specific frame)")
        print("  python framsat1_decoder.py --listen        (Listen live to GNU Radio on port 52001)")
```

Listing 1: Reference script (`framsat1_decoder.py`) for live and offline FramSat-1 frame decoding.

6 Telemetry Format

The raw payload (starting from byte index 16) contains the mission telemetry data. Detailed equations and structures for extracting subsystem voltages, solar currents, and temperatures will be published in a subsequent revision of this guide.

7 Submitting Reception Reports

Orbit NTNU greatly appreciates reports and raw telemetry recordings from the radio community. If you receive packets from FramSat-1, please submit:

- Your callsign and ground station location (grid square or coordinates).
- UTC timestamp of reception.
- Raw hexadecimal packet string or IQ baseband recording (`.wav` / `.sigmf`).
- Estimated SNR or waterfall screenshot.

Reports can be submitted via email to `contact@orbitntnu.com` or uploaded to the SatNOGS network database under satellite **FramSat-1**.